
Compte rendu de TP

[HAI819I] Moteur de jeu - TP1/TP2

I)

I.1)

Avant tout, en base, j'utilise un ensemble de classes que j'ai écrites pour un projet personnel, Glutony, initialement basé sur glut, mais réécrit pour GLFW et glm. La base de code fournie a donc été presque entièrement retirée, car déjà écrite dans glutony.

Mon code utilise des scenes contenant des arborescences de GameObject, dont les transformations sont appliquées relativement au parent (Comme Unity). Chaque GameObject a une liste de composants, qui peuvent être ajoutés et retirés au runtime. Par exemple, les composants meshRenderer et mesh, qui permettent à un objet d'avoir un maillage et une passe de rendu avec un shader.

Petite précision sur le meshRenderer, il utilise un matériau qui est une classe de stockage d'uniforms, qui dépend d'un shader, et qui se charge de synchroniser les uniforms qu'il contient. Un matériau ne fonctionne que sur le shader pour lequel il est créé, mais un même shader peut être utilisé par plusieurs matériaux différents (comme Unity).

Pour le plan, on construit comme demandé, puis il est affiché:

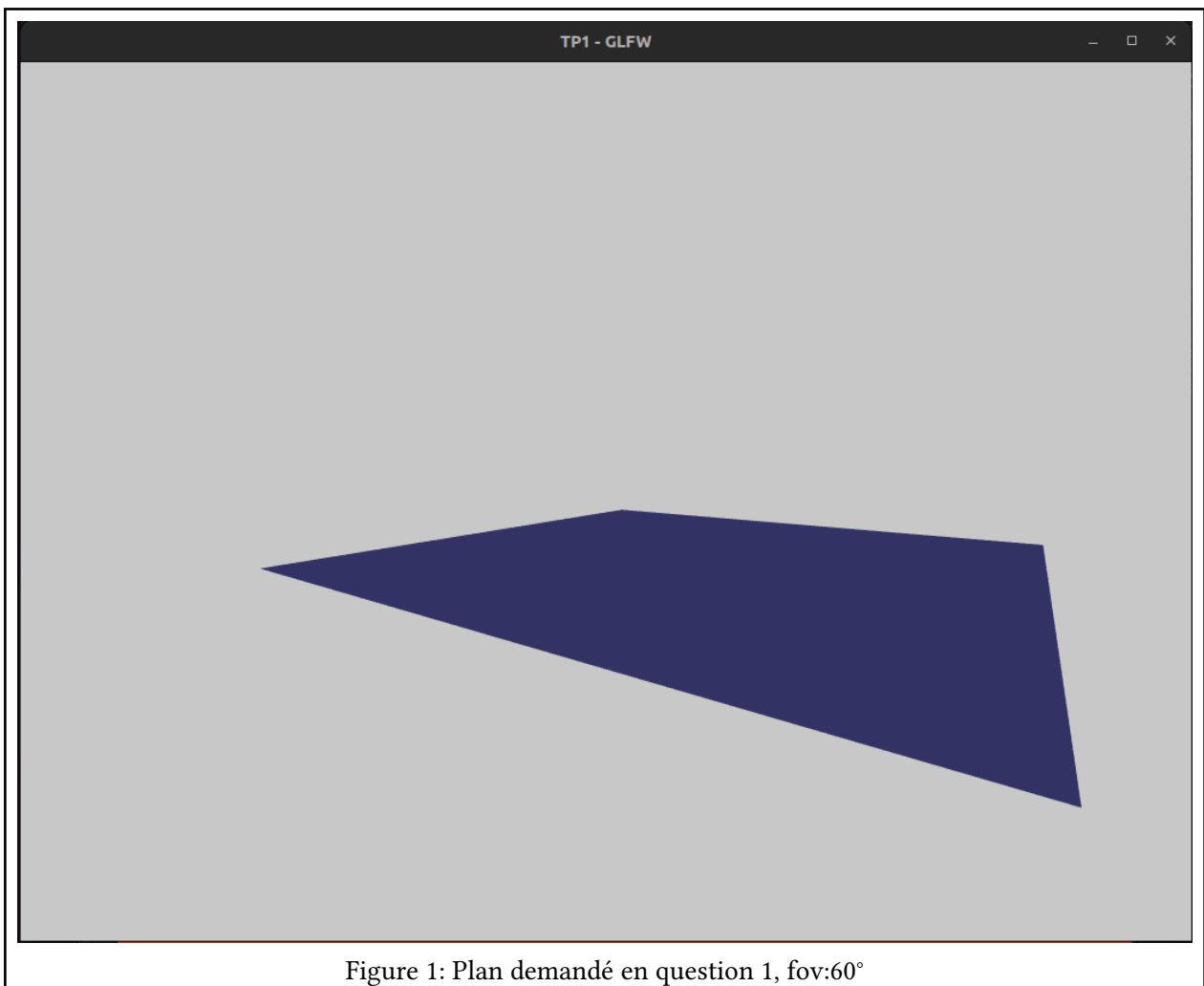


Figure 1: Plan demandé en question 1, fov:60°

La touche g permet d'activer des contrôles FPS [Z/Q/S/D] [E:UP] [A:DOWN]. La direction de mouvement est relative à la rotation sur l'axe Y seulement, pour contraindre horizontalement.

MeshRenderer supporte les uvs, si le composant Mesh en déclare, il les envoie sur l'attribut 2 du VAO. En [Figure 2](#) nous pouvons voir les coordonnées de textures affichées telle que $col = \begin{pmatrix} uv_u \\ uv_v \\ 0.4 \end{pmatrix}$

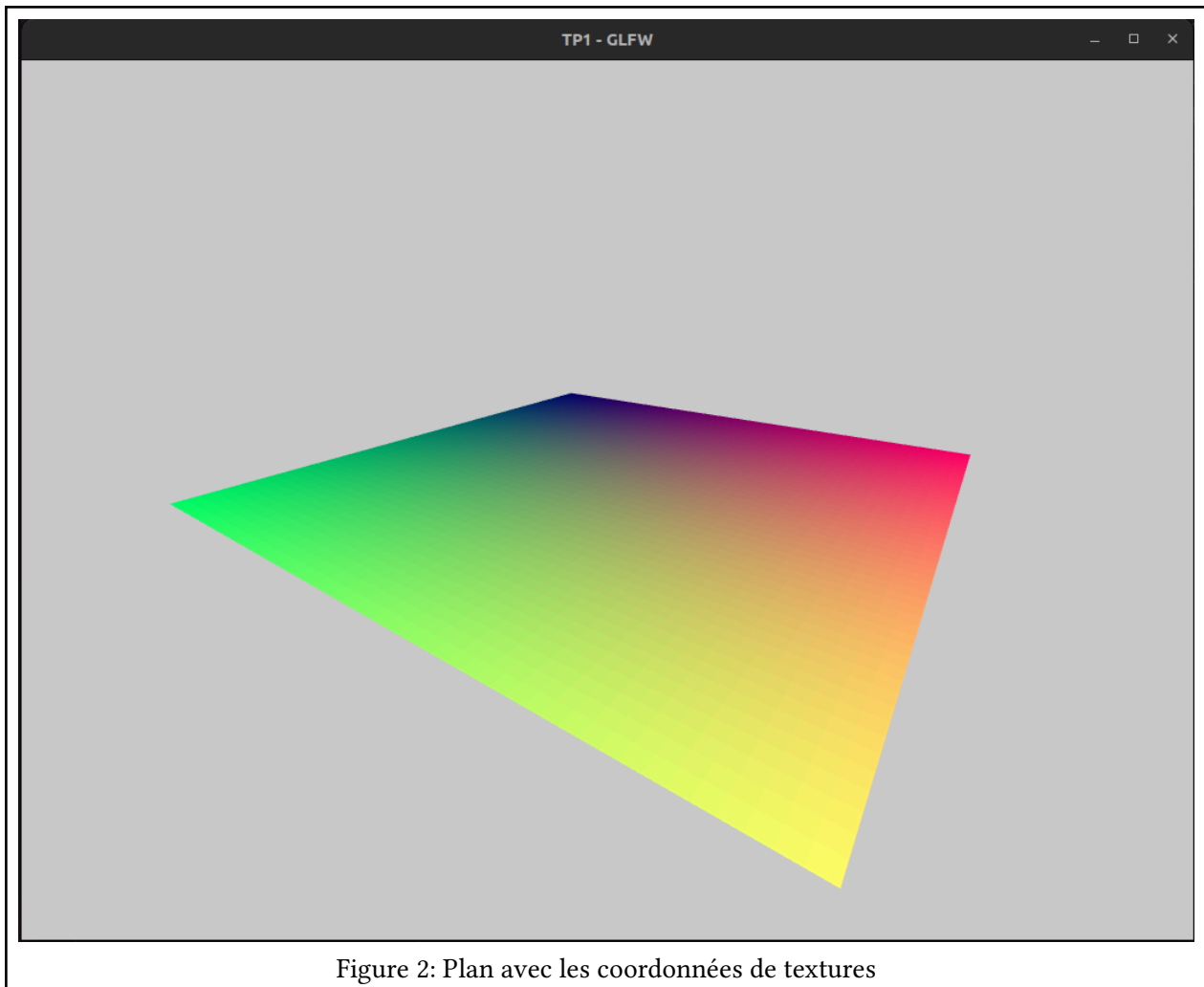


Figure 2: Plan avec les coordonnées de textures

I.2)

Pour l'altitude, un bruit aléatoire continu et cohérent me semblait plus utile (et cela servira sûrement plus tard) donc j'ai écrit un composant `MeshNoiseDeformPerlinHeight`, qui applique un bruit de Perlin(3D) à la hauteur d'un maillage, en fonction des coordonnées objet de ses vertices. la [Figure 3](#) montre le resultat, avec une texture en plus.

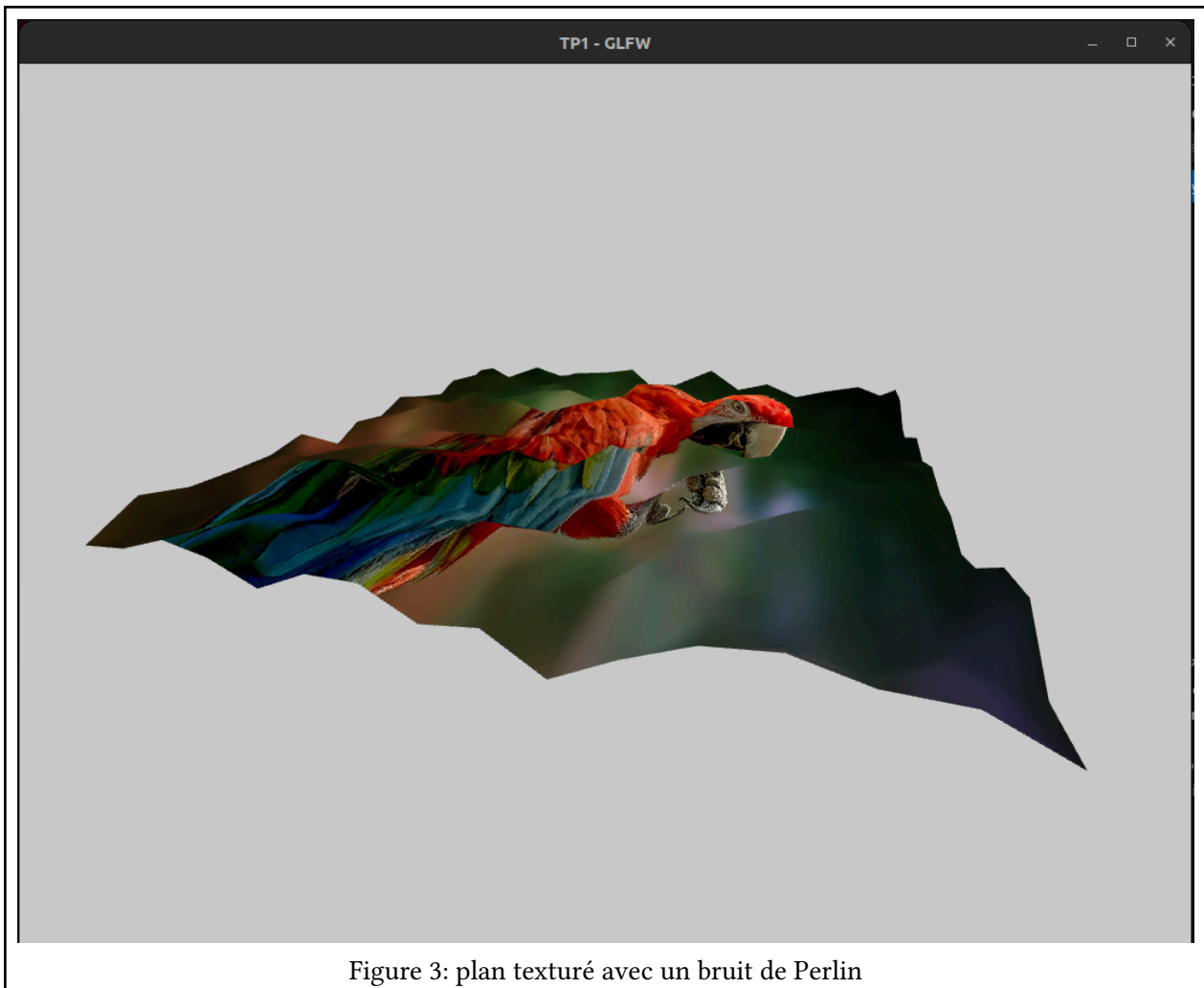


Figure 3: plan texturé avec un bruit de Perlin

II)

II.1)

II.1.a)

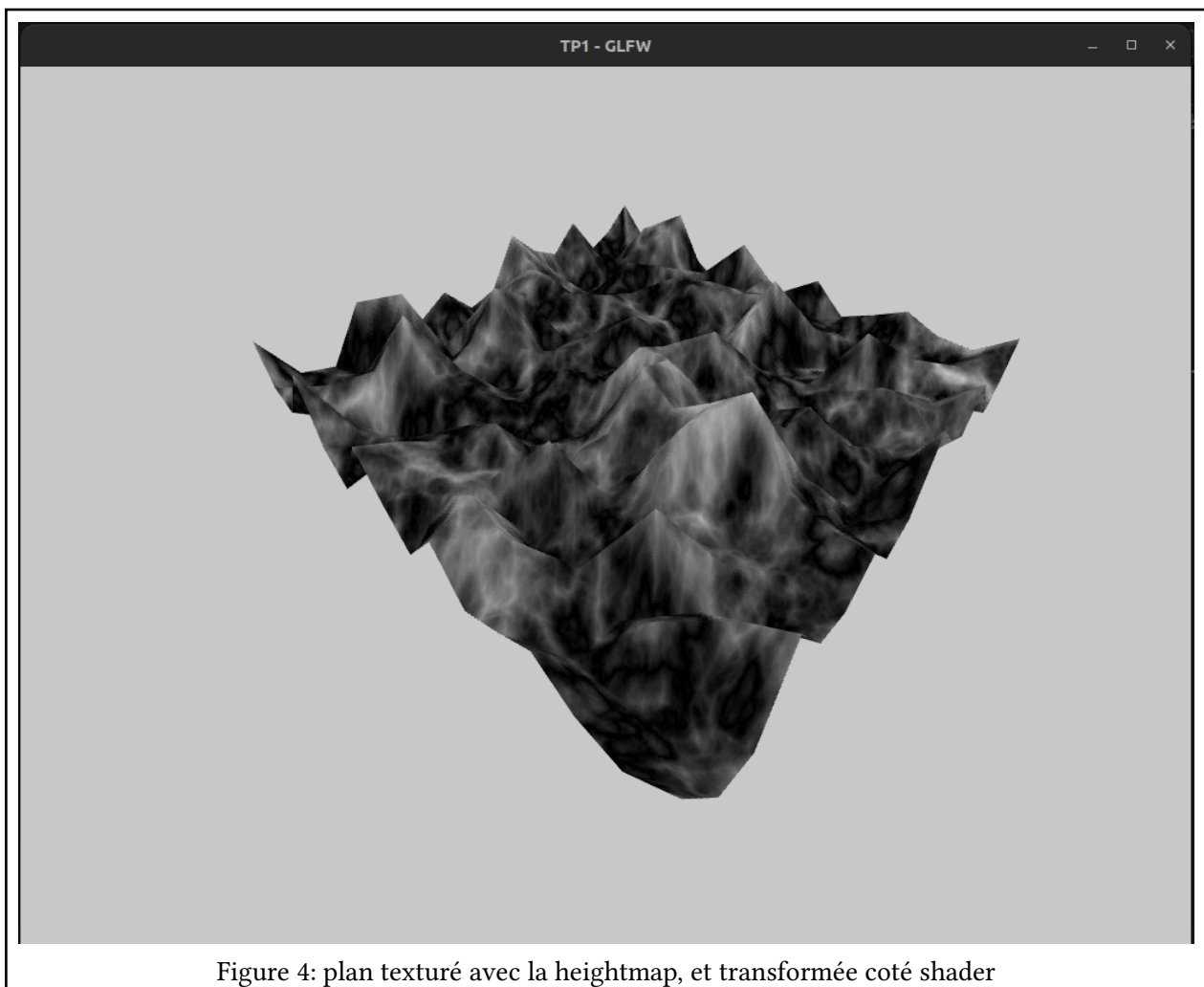


Figure 4: plan texturé avec la heightmap, et transformée coté shader

On multiplie un canal de la texture par 2 et on retranche 1. cela permet de rester centré en zéro.

II.1.b)

on fait un interpolation en trois points, de sorte à ce que chaque texture ai un intervalle de hauteur assigné, et une ou deux zones de transition, ou une interpolation est faite, la [Figure 5](#) illustre le choix de couleur et la [Figure 6](#) montre le résultat.

$$P(y) = y > -0.6 \wedge y < -0.2 \vee y > 0.2 \wedge y < 0.6$$

$$c_0 = t_0[\text{uv}].\text{rgb}$$

$$c_1 = t_1[\text{uv}].\text{rgb}$$

$$c_2 = t_2[\text{uv}].\text{rgb}$$

$$l_0 = \text{lerp}(\text{remap}(-0.6, -0.2, y), c_0, c_1)$$

$$l_1 = \text{lerp}(\text{remap}(0.2, 0.6, y), c_1, c_2)$$

$$c = \neg P(y)?$$

$$y \geq 0.6?$$

$$c_2$$

$$: y \leq -0.6?$$

$$c_0$$

$$: c_1$$

$$: y > 0.2 \wedge y < 0.6?$$

$$l_1$$

$$: l_0$$

Figure 5: Pseudo code du choix de couleur en fonction de la hauteur

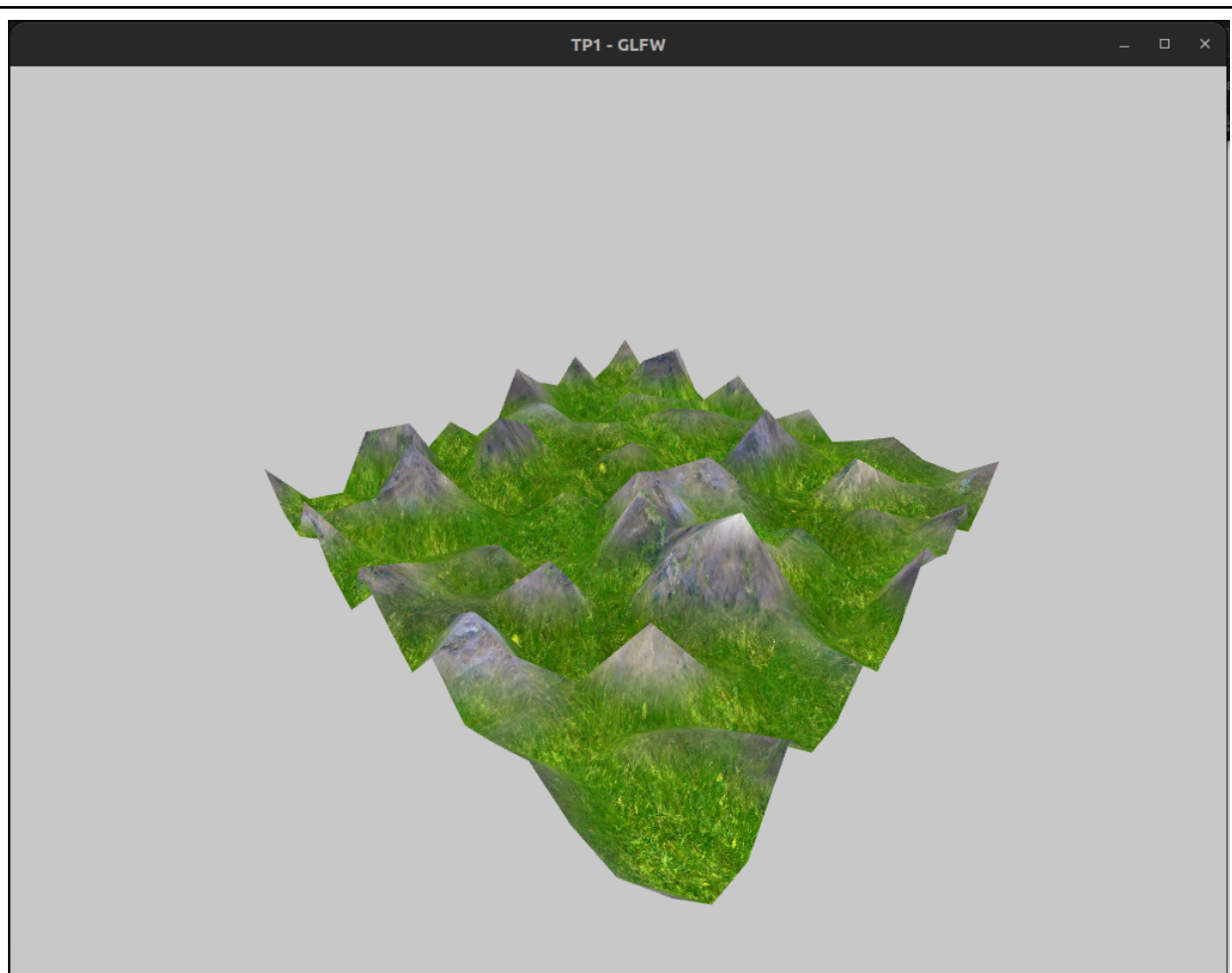
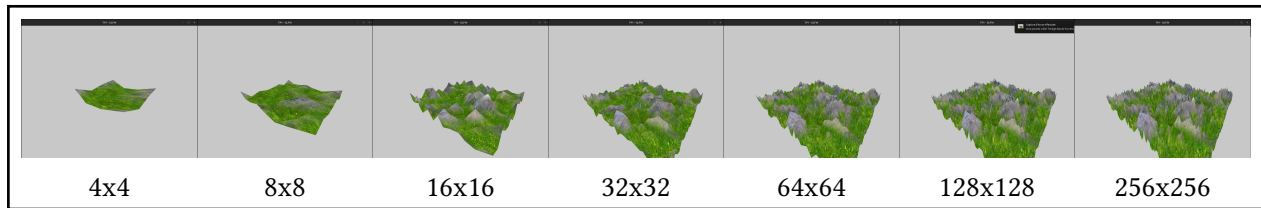


Figure 6: plan qui subit la heightmap et texturé avec nos trois textures

II.1.c)



Vue 1: Terrain à différentes résolutions

J'utilise une classe pour gérer les inputs désormais. Elle est instantiée dans la scène, et surveille les entrées enregistrées, pour distinguer les touches pressées des touches maintenues.

II.2)

Fait sur le TP1. La souris contrôle l'angle de la caméra.

II.3)

```
if (m_inputProcessor.queryKey(window, GLFW_KEY_UP))
{
    m_orbitSpeed += 5.0f;
    std::cout << "Orbit speed : " << m_orbitSpeed << std::endl;
}
if (m_inputProcessor.queryKey(window, GLFW_KEY_DOWN))
{
    m_orbitSpeed -= 5.0f;
    std::cout << "Orbit speed : " << m_orbitSpeed << std::endl;
}

if (m_inputProcessor.queryKey(window, GLFW_KEY_C))
{
    if (m_fpsControl)
    {
        m_fpsControl = false;
        std::cout << "FPS Camera control disabled" << std::endl;
    }
    m_orbitMode = !m_orbitMode;
    std::cout << "orbitmode : " << (m_orbitMode ? "enabled" : "disabled") <<
std::endl;
    if (m_orbitMode)
    {
        glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_NORMAL);
    }
}

if (m_orbitMode)
{
    m_orbitYangle += m_orbitSpeed * ((float)deltaTime);
    m_orbitYangle = m_orbitYangle > 360.0f ? m_orbitYangle - 360.0f : m_orbitYangle;
    m_orbitYangle = m_orbitYangle < 0.0f ? m_orbitYangle + 360.0f : m_orbitYangle;
    vec3 nCamPos = glm::quat(glm::radians(glm::vec3(0.0, m_orbitYangle, 0.0f))) *
m_orbitPos;
    m_camera.m_position = nCamPos;
    m_camera.m_orientation = glm::vec3(-45.0f, m_orbitYangle + 180.0f, 0.0f);
}
```

Extrait 1: Mode orbite

L'Extrait 1 est ce qui me permet d'avoir une orbite. On applique une rotation à un vecteur de position pour la camera, et on compense la rotation pour observer l'origine. Le tout en fonction d'un mode d'affichage. On peut aussi voir comment j'interroge mon inputProcessor. Une image n'était pas pertinente, puisque le résultat est visible qu'en vidéo ou temps réel. Cependant, en exécutant mon code [Branche github Rendu TP2 - MC, https://github.com/HugoGellida/Glutony_lol/tree/Rendu-TP2---MC], c'est observable en pressant C. (M et P réduisent la résolution et construisent le mesh, et G passe en mode FPS, ou A et E permettent de monter et descendre, et ZQSD permettent le déplacement horizontal), en mode FPS, la souris disparaît.